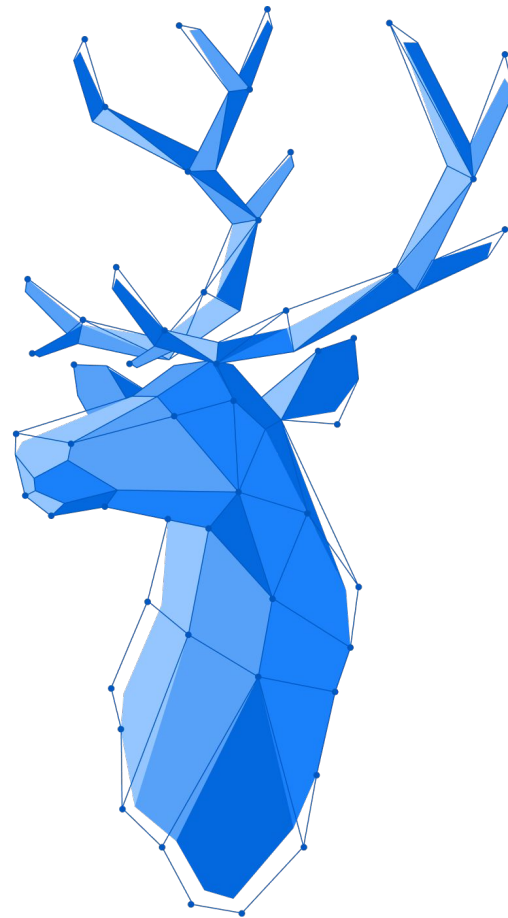




Komplexe Komponenten

Von Gerald Koch



Möglichkeiten

Composite

- Kapselt die neue Komponente und versteckt deren API.
- Interfaces wie HasLabel müssen erneut implementiert werden, oder können über den Umweg `getContent()` verwendet werden

Ableitung

- Leitet eine Vaadin Komponente ab und überschreibt Ihre Funktionalität. Die originale API bleibt bestehen.
- Die Benutzung der Api des abgeleiteten Elementes muss eventuell überschrieben werden, um z.B.:
`getElement().getStyle()` an das richtige Element zu bringen, steht dann aber für die äußere Komponente nicht mehr zur Verfügung

Das Interface HasValue

- Nötig wenn Komponente in Binder verwendet werden soll
- Besitzt folgende Methoden:
 - `V getValue()`
 - `setValue(V)`
 - `Registration addValueChangeListener(ValueChangeListener<? super E>)`
 - `setReadOnly(boolean)`
 - `boolean isReadOnly()`
 - `setRequiredIndicatorVisible(boolean)`
 - `boolean isRequiredIndicatorVisible()`
- Optional können auch noch weitere Methoden wie `clear()` oder `isEmpty()` überschrieben werden
- Alle Vaadin-Eingabekomponenten implementieren `HasValue`
- Der Value `v` kann auch komplexe Formen annehmen